S32M276 PMSM FOC FIRMWARE

Position-Control Tuning Guide

FreeMASTER Variable Watch commissioning procedure for relative multi-turn mechanical position control.

Motor: Faulhaber 3216 W 012 BXT H
Encoder: IEF3-4096L-128, 512 quadrature counts/revolution
Initial condition: Mechanically unloaded
Requested later ceiling: 2000 RPM
Document date: 22 July 2026

Bench commissioning guide. Calculated starting values are not a claim of hardware validation.

Contents

Diagnostic flags are nonzero

FreeMASTER cannot find the variables

11. End-of-session safe shutdown

12. Final recommended starting-value card

1. Purpose and assumptions

This document is the step-by-step procedure for the first hardware tuning of the custom relative position controller.

This procedure assumes:

- Faulhaber 3216 W 012 BXT H motor;
- motor mechanically unloaded;
- encoder resolution of 128 lines/revolution, or 512 quadrature counts/revolution;
- 7 motor pole pairs;
- position targets entered in mechanical degrees;
- requested eventual position-mode ceiling of 2000 RPM;
- tuning performed through the FreeMASTER Variable Watch window.

Position is relative, not absolute. After reset, the firmware does not know an absolute machine position and does not perform homing. One encoder count is:

```
360 degrees / 512 counts = 0.703125 mechanical degree/count
```

The requested 2000 RPM ceiling is:

```
2000 RPM x 2*pi / 60 = 209.4395 mechanical rad/s
```

The firmware-derived mechanical safety ceiling is approximately 543.914 rad/s (5194 RPM), calculated from `WEL_MAX`, the existing 0.98 application factor, and 7 pole pairs. Therefore, 2000 RPM is below the firmware ceiling. It is still too aggressive for the first position test. Start at 100 RPM and raise the limit only after the lower-speed stages pass.

Authoritative configuration locations:

- Pole pairs: `src/config/PMSM_appconfig.h:25`
- Electrical speed scale: `src/config/PMSM_appconfig.h:34`
- Encoder lines/revolution: `src/config/PMSM_appconfig.h:99`
- Position-loop period (0.001 s): `src/main.c:148`
- Firmware safety-speed derivation: `src/main.c:1039-1048`
- Existing 1 kHz slow-loop call site: `src/main.c:1808-1828`

This guide provides conservative engineering starting values. They are not hardware-validated values. Final values must be established from real bench results.

2. Safety rules

Before applying motor-bus power:

1. Secure the unloaded motor so its body and wiring cannot rotate or be pulled into the rotor.
2. Keep the shaft completely free of tools, clothing, wiring, and personnel.
3. Have an independent means of removing motor-bus power. FreeMASTER is not an emergency stop.

4. Use a current-limited supply where the hardware permits it. Do not weaken existing firmware current, voltage, temperature, or speed protections.

5. Confirm the original encoder speed-control mode still works with position control disabled.
6. Keep `PC Ki` at zero throughout the first unloaded tests.
7. Begin with +5 and -5 degree targets. Do not begin with a full revolution or a 2000 RPM move.
8. If the motor moves when position mode is enabled but before a target is sent, remove power and investigate. Enable is designed to capture the current position and request zero speed.

3. FreeMASTER preparation

1. Build the latest `Debug_FLASH` firmware so the ELF contains the custom position-control globals.
2. Make sure the FreeMASTER project uses the newly built ELF, not an older ELF.
3. Begin from a fresh controller reset. This restores the unlisted software-limit fields to their disabled defaults at `src/main.c:1061-1064`.
4. Add the variables listed in Sections 4 and 5 to Variable Watch.
5. Treat every `g_positionCommandStaged` and `g_positionCommand` entry in this guide as writable.
6. Treat every `g_positionStatus` entry as read-only. Never use a status field as a command.

The global objects are declared at:

- `g_positionCommandStaged`: `src/main.c:252`
- `g_positionCommand`: `src/main.c:253`
- `g_positionStatus`: `src/main.c:254`

4. Required writable variables

Use the suggested display name in FreeMASTER so commands, settings, and status are easy to distinguish.

4.1 Tuning values

Suggested FreeMASTER name	Exact symbol	First-test value	Units	Purpose	Source definition
`PC Tune - Kp`	`g_positionCommandStaged.tuning.kpMechanicalRadPerSecPerCount`	`0.0200`	mechanical rad/s/count	Converts position error into a proportional speed request.	`src/position_control.h:63`; copied at `src/main.c:1001-1002`
`PC Tune - Ki`	`g_positionCommandStaged.tuning.kiMechanicalRadPerSecPerCountSec`	`0.0`	mechanical rad/s/(count*s)	Integrates persistent position error. Keep zero for unloaded commissioning.	`src/position_control.h:64`; copied at `src/main.c:1003-1004`
`PC Tune - Integral Limit`	`g_positionCommandStaged.tuning.integratorLimitMechanicalRadPerSec`	`1.0472`	mechanical rad/s	Limits integral contribution to 10 RPM. It must be greater than zero before Ki can be nonzero.	`src/position_control.h:65`; copied at `src/main.c:1005-1006`
`PC Tune - Maximum Speed`	`g_positionCommandStaged.tuning.maxMechanicalRadPerSec`	`10.4720`	mechanical rad/s	Limits initial position-mode speed to 100 RPM.	`src/position_control.h:66`; copied at `src/main.c:1007-1008`

Suggested FreeMASTER name	Exact symbol	First-test value	Units	Purpose	Source definition
PC Tune - Maximum Acceleration	g_positionCommandStaged.tuning.maxAccelMechanicalRadPerSec2	20.9440	mechanical rad/s^2	Limits acceleration to 200 RPM/s.	src/position_control.h:67; copied at src/main.c:1009-1010
PC Tune - Maximum Deceleration	g_positionCommandStaged.tuning.maxDecelMechanicalRadPerSec2	20.9440	mechanical rad/s^2	Limits deceleration to 200 RPM/s. Lower it if DC-bus voltage rises excessively during stopping.	src/position_control.h:68; copied at src/main.c:1011-1012
PC Tune - Position Tolerance	g_positionCommandStaged.tuning.positionToleranceCounts	1	encoder count	Requests zero position speed when absolute error is 1 count or less. One count is 0.703125 degree.	src/position_control.h:69; copied at src/main.c:1013-1014
PC Tune - Stopped Speed	g_positionCommandStaged.tuning.stoppedMechanicalRadPerSec	0.1000	mechanical rad/s	Shaft must be below this speed, about 0.955 RPM, before atTarget can qualify.	src/position_control.h:70; copied at src/main.c:1015-1016
PC Tune - Target Dwell	g_positionCommandStaged.tuning.atTargetDwellSec	0.2500	second	Position and speed must remain qualified for 250 ms before atTarget becomes 1.	src/position_control.h:71; copied at src/main.c:1017-1018

Why PC Kp = 0.0200 is the recommended starting point:

- It gives an equivalent outer position-loop gain of approximately 1.63 per second with the 512-count encoder.
- A 5 degree error becomes approximately 7 counts and requests only about 0.142 rad/s, or 1.36 RPM.
- A one-revolution error requests approximately 10.24 rad/s, close to the initial 100 RPM speed limit.
- It keeps the outer-loop response below the existing 1 Hz speed-loop bandwidth.

The actual P/PI calculation and limiting are implemented at src/position_control.c:701-760. Tuning validation is at src/position_control.c:195-245.

4.2 Target values

Suggested FreeMASTER name	Exact symbol	Required value	Units	Purpose	Source definition
PC Target - Units	g_positionCommandStaged.targetUnits	1	enum	Selects degrees. 0 means counts and 1 means degrees. Leave it at 1 for this guide.	src/main.c:163; enum values at src/main.c:152-156; copied at src/main.c:1029
PC Target - Degrees	g_positionCommandStaged.targetDegrees	0.0 initially; then +5.0 or -5.0	mechanical degree	The next relative multi-turn position target.	src/main.c:162; copied at src/main.c:1028

Because the encoder has 0.703125 degree/count resolution, a requested 5 degrees is rounded to 7 counts, or 4.921875 degrees. This is expected and is not a tuning error.

4.3 Command counters

Suggested FreeMASTER name	Exact symbol	Value to write	Purpose	Source definition	Processing location
`PC Command - Apply Tuning`	`g_positionCommand.applySequence`	current value + 1	Atomically validates and activates all staged tuning values. Write it last after changing tuning.	`src/main.c:168`	`src/main.c:1185-1197`
`PC Command - Enable`	`g_positionCommand.enableSequence`	current value + 1	Enables position ownership, captures the current position, clears motion state, and waits for a later target.	`src/main.c:169`	`src/main.c:1208-1215`
`PC Command - Disable`	`g_positionCommand.disableSequence`	current value + 1	Disables position control and safely releases its speed-reference ownership.	`src/main.c:170`	`src/main.c:1173-1179`
`PC Command - Send Target`	`g_positionCommand.setTargetSequence`	current value + 1	Accepts the staged target value and units. Write it last after changing the target.	`src/main.c:172`	`src/main.c:1217-1241`

Command counters are events, not ON/OFF variables. At a fresh reset they start at zero (`src/main.c:1073-1077`). A normal sequence is:

Event	Enable counter	Disable counter	Apply counter	Target counter
Reset	0	0	0	0
Apply first tuning	0	0	1	0
Enable	1	0	1	0
Send first target	1	0	1	1
Disable	1	1	1	1
Enable again	2	1	1	1
Send another target	2	1	1	2

Do not set the enable counter to zero to disable. Changing a processed command counter in either direction can be interpreted as another command. Always use the separate disable counter and monotonically increment each counter.

5. Required read-only status variables

Suggested FreeMASTER name	Exact symbol	Expected condition	Purpose	Source definition/update
`PC Encoder - Wrapped Count`	`encoderPospe.correctedWrappedMechanicalCount`	`0` through `511`	Direct corrected encoder count used for the manual direction and one-revolution scaling check.	Field at `src/pospe_sensor.h:77`; global object at `src/main.c:234`; updated at `src/pospe_sensor.c:66`

Suggested FreeMASTER name	Exact symbol	Expected condition	Purpose	Source definition/update
Position-Control Tuning Guide				S32M276 / Faulhaber 3216
`PC Status - Control Permitted`	`g_positionStatus.controlPermitted`	`1` before enable	Confirms the application is running in encoder-based speed control.	`src/main.c:196` ; policy at `src/main.c:1094-1100` ; update at `src/main.c:1149`
`PC Status - Encoder Valid`	`g_positionStatus.encoderValid`	`1` before enable	Confirms the position module has valid encoder data.	`src/main.c:197` ; update at `src/main.c:1150`
`PC Status - Enabled`	`g_positionStatus.enabled`	`1` after enable	Confirms the enable request was accepted.	`src/main.c:194` ; update at `src/main.c:1147`
`PC Status - Ownership Active`	`g_positionStatus.ownershipActive`	`1` after enable	Confirms the position controller currently owns the existing speed reference.	`src/main.c:195` ; update at `src/main.c:1148` ; ownership write at `src/main.c:1249-1268`
`PC Status - Target Armed`	`g_positionStatus.targetArmed`	`0` immediately after enable; `1` after target	Confirms a later explicit target has been accepted.	`src/main.c:198` ; update at `src/main.c:1151`
`PC Status - Actual Position Deg`	`g_positionStatus.actualPositionDegrees`	Observe	Relative multi-turn mechanical position.	`src/main.c:178` ; update at `src/main.c:1126`
`PC Status - Position Error Counts`	`g_positionStatus.positionErrorCounts`	Trends toward `0` , normally finishes within +/-1	Target minus actual position.	`src/main.c:180` ; update at `src/main.c:1128`
`PC Status - Measured Speed`	`g_positionStatus.measuredMechanicalRadPerSec`	Near `0` at rest	Required to judge settling and stopped-speed qualification.	`src/main.c:184` ; update at `src/main.c:1134-1135`
`PC Status - Integral Contribution`	`g_positionStatus.integratorMechanicalRadPerSec`	`0` while Ki is zero	Shows integral buildup when Ki is introduced later.	`src/main.c:185` ; update at `src/main.c:1136-1137`
`PC Status - At Target`	`g_positionStatus.atTarget`	Eventually `1`	Confirms error, stopped speed, and dwell requirements are all met.	`src/main.c:199` ; update at `src/main.c:1152`
`PC Status - Tuning Accepted`	`g_positionStatus.applyAccepted`	`1` after every apply	Indicates the latest tuning snapshot passed validation.	`src/main.c:203` ; written at `src/main.c:1193-1196`
`PC Status - Tuning Result`	`g_positionStatus.tuningResult`	`1` after apply	Identifies why a tuning snapshot was rejected.	`src/main.c:190` ; update at `src/main.c:1145` ; enum at `src/position_control.h:23-31`
`PC Status - Diagnostic Flags`	`g_positionStatus.diagnosticFlags`	`0x00000000`	Reports encoder, configuration, permission, or numeric problems. Display in hexadecimal.	`src/main.c:189` ; update at `src/main.c:1144` ; bits at `src/position_control.h:35-43`

Do not write any `g_positionStatus` variable from FreeMASTER, even though it resides in RAM.

6. Exact first-tuning procedure

Follow these steps in order.

Step 1: Start in the original encoder speed-control mode

1. Keep position control disabled.
2. Enter the application's normal run state.
3. Select existing `speedControl` mode, encoder sensor feedback, and `encoder1` position feedback.

4. Verify:


```
PC Status - Control Permitted = 1
PC Status - Encoder Valid     = 1
PC Status - Diagnostic Flags  = 0x00000000
```

The exact permission conditions are at `src/main.c:1094-1100`.

Step 2: Verify position direction with motor output disabled

1. With PWM/motor output safely disabled, rotate the shaft manually in the intended positive direction.
2. Observe `PC Encoder - Wrapped Count`. It must stay in the range 0 through 511 and change consistently with rotation.
3. Cross the count wrap in both directions. Forward wrap should behave as `511 -> 0`; reverse wrap should behave as `0 -> 511`.
4. Rotate one complete mechanical revolution and confirm exactly 512 count transitions occur.
5. When the application later enters the permitted run state, confirm `PC Status - Actual Position Deg` uses the same positive direction.

If the sign is wrong, do not use negative Kp or Ki. Negative gains are rejected, and changing gain sign would not safely correct the encoder/drive sign convention. Stop and correct the encoder direction integration before powered position tests.

Step 3: Write the initial tuning

Write all of these staged values:

```
PC Tune - Kp                   = 0.0200
PC Tune - Ki                   = 0.0
PC Tune - Integral Limit       = 1.0472
PC Tune - Maximum Speed        = 10.4720
PC Tune - Maximum Acceleration = 20.9440
PC Tune - Maximum Deceleration = 20.9440
PC Tune - Position Tolerance   = 1
PC Tune - Stopped Speed        = 0.1000
PC Tune - Target Dwell         = 0.2500
PC Target - Units              = 1
PC Target - Degrees            = 0.0
```

Increment `PC Command - Apply Tuning` once. Then verify:

```
PC Status - Tuning Accepted = 1
PC Status - Tuning Result   = 1
```

If tuning is rejected, do not enable. Correct the rejected values and increment the apply counter again. A rejected staged configuration does not partially replace the previous active configuration.

Tuning-result values:

```
0 = not applied
1 = accepted
2 = null configuration
3 = NaN, infinity, or another invalid numeric value
4 = invalid range or limit
5 = inconsistent software limits
```

Step 4: Enable without commanding motion

1. Reconfirm `Control Permitted = 1`, `Encoder Valid = 1`, and diagnostics are zero.
2. Increment `PC Command - Enable` once.
3. Do not write the target or target counter at the same time.
4. Verify:

```
PC Status - Enabled          = 1
PC Status - Ownership Active = 1
PC Status - Target Armed     = 0
PC Status - Measured Speed   approximately 0
```

The shaft should not move. Enable captures the current position as the internal target, clears the integral and rate-limiter states, and leaves motion unarmed. This behavior is implemented at `src/position_control.c:412-445`.

If the shaft moves on enable, increment the disable counter and remove motor-bus power. Do not continue tuning.

Step 5: Command +5 degrees

1. Set `PC Target - Units = 1`.
2. Set `PC Target - Degrees = 5.0`.
3. Increment `PC Command - Send Target` once.
4. Confirm `PC Status - Target Armed = 1`.
5. Observe actual position, position error, measured speed, existing Iq feedback/request, DC-bus voltage, and existing fault status.
6. Wait for `PC Status - At Target = 1`.

Expected behavior:

- Actual position increases toward approximately 4.92 degrees because of encoder quantization.
- Position error approaches zero and finishes within +/-1 count.
- Motion is slow and smooth.
- No repeated hunting, buzzing, overcurrent, or DC-bus overvoltage occurs.

Step 6: Command -5 degrees

Set `PC Target - Degrees = -5.0`, then increment the target counter again. This is an absolute target in the current relative coordinate system, so the move from approximately +5 to -5 degrees is approximately 10 degrees.

Confirm correct reverse direction and the same stable settling behavior.

Step 7: Disable and re-enable

1. Increment `PC Command - Disable` once.
2. Verify `Enabled = 0` and `Ownership Active = 0`.
3. Confirm the original application path remains usable.
4. To enable again, increment the enable counter again; do not reuse its previous value.
5. After re-enable, wait for `Target Armed = 0`, then send a new target by incrementing the target counter.

Disable and permission loss clear the target-armed state, integrator, motion limiter, and generated speed request. A new target is required after every successful re-enable.

7. Kp tuning procedure

Keep Ki at zero. Use alternating +5 and -5 degree targets and disable position control before large tuning changes.

Recommended Kp progression:

```
0.0100 -> 0.0200 -> 0.0300 -> 0.0400
```

At each value:

1. Write Kp.
2. Increment the apply counter.
3. Confirm tuning acceptance.
4. Enable if necessary.
5. Send +5 and -5 degree targets.
6. Wait for complete settling before the next command.

Keep the lowest Kp that gives acceptable settling and holding behavior.

Reduce Kp, normally by 25% to 50%, if any of these occur:

- repeated overshoot;
- hunting around the target;
- audible buzzing or chatter;
- alternating position error without settling;
- repeated current saturation in the existing current/speed displays;
- the existing speed loop cannot track the requested response.

Do not progress above `0.0400` during initial unloaded commissioning without reviewing recorded position, speed, and current behavior. With this encoder, `0.0400` already gives an equivalent outer-loop gain of approximately 3.26 per second, which is a substantial fraction of the existing speed-loop bandwidth.

8. Ki tuning procedure

For the unloaded motor, the recommended Ki is `0.0`. Encoder quantization and the existing speed loop normally make integral position control unnecessary during the first test.

Only introduce Ki after P-only behavior is stable and only if a persistent error remains under the intended real load.

Conservative Ki progression:

```
0.00000 -> 0.00025 -> 0.00050 -> 0.00100
```

Keep the integral limit at `1.0472` mechanical rad/s (10 RPM) during these tests. Monitor `PC Status - Integral Contribution`.

Immediately return Ki to zero if:

- the motor develops slow oscillation;
- overshoot grows after repeated moves;
- the integral contribution stays at its positive or negative limit;
- the motor creeps or hunts near the target;
- current remains elevated after the target is reached.

The integrator is cleared inside the position tolerance and on disable, invalid feedback, or permission loss. Anti-windup inhibits integration when limiting prevents motion in the error direction (`src/position_control.c:713-760`).

9. Increasing the speed ceiling toward 2000 RPM

Do not jump directly from the first test to 2000 RPM. Use the following progression only after direction, Kp response, stopping, disable, and fault behavior have passed at the preceding stage.

Stage	Maximum speed	FreeMASTER value	Suggested acceleration/deceleration	FreeMASTER value
First test	100 RPM	`10.4720` rad/s	200 RPM/s	`20.9440` rad/s^2
Intermediate 1	250 RPM	`26.1799` rad/s	300 RPM/s	`31.4159` rad/s^2
Intermediate 2	500 RPM	`52.3599` rad/s	500 RPM/s	`52.3599` rad/s^2
Intermediate 3	1000 RPM	`104.7198` rad/s	500 RPM/s	`52.3599` rad/s^2
Requested ceiling	2000 RPM	`209.4395` rad/s	500 RPM/s initially	`52.3599` rad/s^2

At every stage:

1. Disable position control.
2. Change maximum speed, acceleration, and deceleration.
3. Increment the apply counter and confirm acceptance.
4. Re-enable and send a new small target first.
5. Increase target distance only after the small target remains stable.
6. Monitor existing Iq, speed tracking, DC-bus voltage, and fault status.

The configured maximum speed is only a clamp; a small target and moderate Kp may never request that speed. Reaching 2000 RPM requires a sufficiently large position error. Do not create a large multi-turn step merely to prove the clamp until safe stopping distance and DC-bus regeneration have been verified.

If the DC-bus voltage rises during stopping, reduce the deceleration value. A smaller deceleration number creates a slower stop and normally reduces regenerative stress.

10. Common problems and exact responses

Enable does nothing

Check in this order:

1. `Control Permitted` must be 1.
2. `Encoder Valid` must be 1.
3. `Diagnostic Flags` should be zero.
4. The enable counter must be different from the last successfully processed enable value.
5. The tuning must have been accepted.

Position control is permitted only in application `run`, existing `speedControl`, encoder sensor, and `encoder1` feedback (`src/main.c:1094-1100`).

Enable was incremented while Control Permitted was zero

The enable command remains pending because it is processed only when permission becomes true. This can cause a later unexpected enable.

Before making control permitted:

- restore the enable counter to the last successfully processed enable value, if that value is known; or
- with motor-bus power removed, reset the controller so all command counters restart at zero.

Do not increment enable and disable together. Disable has priority for one 1 ms boundary, but the unprocessed enable can remain pending and execute on the following boundary.

Motor enables but does not move after a target

Check:

1. `Target Units = 1`.
2. Target degrees is finite and nonzero.
3. The target counter was incremented after writing the target.
4. `Target Armed = 1`.
5. Kp is greater than zero.
6. `Ownership Active = 1`.
7. Position error is outside the configured tolerance.

If enable and target counters were changed before the same 1 ms processing boundary, the target can execute automatically on the next boundary. For predictable commissioning, wait until `Enabled = 1` and `Target Armed = 0` before writing and sending the target.

Tuning is rejected

Use `Tuning Result`:

- Result 3: check for NaN, infinity, or corrupted floating-point input.
- Result 4: Kp/Ki or nonnegative fields may be negative; speed/acceleration/deceleration may be zero; Ki may be nonzero with a zero integral limit; or maximum speed may exceed the firmware ceiling.

- Result 5: enabled negative software limit is not below the enabled positive limit.

The previous active tuning remains active. Correct all staged values, then increment the apply counter again.

Direction is wrong

Disable immediately. Do not make Kp negative; negative Kp is rejected. Verify mechanical encoder direction and the existing encoder speed-control direction with motor output disabled before continuing.

Motor overshoots, hunts, buzzes, or chatters

1. Increment disable.
2. Set Ki to zero.
3. Reduce Kp by 50%.
4. Reduce maximum speed and acceleration.
5. Apply tuning and repeat only a +/-5 degree test.

One encoder count is 0.703125 degree, so sub-count mechanical regulation is impossible. Some small final quantization is normal.

At Target never becomes one

Check that position error reaches +/-1 count and measured speed falls below 0.1000 rad/s for at least 0.25 second.

If the shaft is physically settled but encoder/speed noise prevents qualification:

1. Try position tolerance `2` counts.
2. If necessary, try stopped speed `0.2000` rad/s.
3. Keep dwell at `0.2500` second initially.

Do not enlarge these thresholds merely to hide visible motion or oscillation.

Position control stops unexpectedly

Permission loss, encoder invalidity, stop, fault, alignment, or leaving encoder speed mode disables position ownership. Check `Control Permitted`, `Encoder Valid`, and diagnostic flags. After the condition is corrected, increment enable again and then send a new target.

Diagnostic flags are nonzero

Display the value in hexadecimal:

```
0x01 = invalid controller configuration
0x02 = encoder count outside configured range
0x04 = ambiguous exact half-revolution sample change
0x08 = encoder movement exceeded the physical per-sample limit
0x10 = application permission denied
0x20 = invalid numeric calculation
```

Multiple bits can be set. Do not continue powered testing until the cause is understood and the flag returns to zero.

FreeMASTER cannot find the variables

1. Refresh the S32DS project so `src/position_control.c` is included.
2. Rebuild `Debug_FLASH`.
3. Confirm the build completes successfully.
4. Reload the new ELF/map symbols in FreeMASTER.
5. Reconnect and search for the three global names again.

11. End-of-session safe shutdown

1. Increment `PC Command - Disable` once.
2. Confirm `Enabled = 0` and `Ownership Active = 0`.
3. Stop the original motor-control application using its normal procedure.
4. Remove motor-bus power.
5. Record the tested Kp, Ki, limits, supply settings, target sizes, and observed behavior.
6. Do not copy values into firmware defaults until the complete low-speed and requested-speed progression has passed on hardware.

12. Final recommended starting-value card

```
Motor condition:                unloaded
Target units:                   1 (degrees)
Kp:                             0.0200 mechanical rad/s/count
Ki:                             0.0 mechanical rad/s/(count*s)
Integral limit:                 1.0472 mechanical rad/s (10 RPM)
Maximum speed:                  10.4720 mechanical rad/s (100 RPM)
Maximum acceleration:           20.9440 mechanical rad/s^2 (200 RPM/s)
Maximum deceleration:           20.9440 mechanical rad/s^2 (200 RPM/s)
Position tolerance:             1 count (0.703125 degree)
Stopped-speed threshold:        0.1000 mechanical rad/s (0.955 RPM)
At-target dwell:                0.2500 second
First targets:                  +5.0 degrees, then -5.0 degrees
Requested later speed ceiling:  209.4395 mechanical rad/s (2000 RPM)
```

Command reminder:

```
Apply tuning: increment applySequence
Enable:       increment enableSequence
Send target:  increment setTargetSequence
Disable:      increment disableSequence
```

Never interpret an enable/disable command counter as a Boolean state. The actual state is reported only by `g_positionStatus.enabled` and `g_positionStatus.ownershipActive`.